

Prototype IA

Génération d'un plan de remédiation,
backlog et estimation de charge

Rapport de projet — Analyse IA d'audit informatique

Réalisé par :

Aicha Habib

Date : 28 juin 2026

Table des matières

1. Compréhension du sujet	3
2. Objectif de la solution IA.....	3
3. Données d'entrée utilisées	3
4. Méthode d'analyse des constats	3
5. Critères de criticité et de priorité.....	4
6. Plan de remédiation proposé	4
7. Transformation en backlog.....	5
8. Méthode d'estimation de charge	5
9. Gestion des dépendances et des risques.....	6
10. Roadmap de traitement proposée	7
11. Limites de la solution.....	7
12. Ce que j'ai fait avec l'aide de l'IA	7
13. Ce qui relève de mon travail personnel	8
14. Conclusion et pistes d'amélioration	8
Conclusion	8
Pistes d'amélioration.....	8

1. Compréhension du sujet

L'objectif de cet exercice est de concevoir une solution basée sur l'intelligence artificielle capable d'analyser automatiquement des constats techniques issus d'un audit informatique afin de produire un plan de remédiation exploitable.

La solution doit permettre d'identifier les principaux problèmes, de les classer selon leur criticité, de proposer des actions correctives, de générer un backlog priorisé, d'estimer la charge de travail, d'identifier les dépendances entre les tâches et de construire une roadmap de traitement réaliste.

2. Objectif de la solution IA

Le prototype développé a pour objectif d'automatiser l'analyse d'un fichier d'audit technique au format CSV. À partir de ces données, le système :

- identifie les catégories de problèmes ;
- attribue un niveau de criticité ;
- définit une priorité (P1 à P4) ;
- propose une action corrective ;
- estime la charge de travail en jours/homme ;
- génère automatiquement un plan de remédiation ;
- construit un backlog priorisé ;
- produit une roadmap de traitement.

Cette approche permet de gagner du temps et d'harmoniser les décisions de priorisation.

3. Données d'entrée utilisées

Les données d'entrée sont stockées dans un fichier `audit.csv`. Chaque ligne représente un constat technique identifié lors d'un audit. Le fichier contient les colonnes suivantes :

Colonne	Description
ID	Identifiant unique du constat
Category	Catégorie du problème (Security, Performance, etc.)
Finding	Titre court du constat
Description	Description détaillée du problème identifié
Component	Composant ou module concerné
Depends_On	Identifiants des constats dont ce constat dépend
Status	Statut actuel du constat (Open, In Progress, Closed)

Les catégories couvrent plusieurs domaines : Security, Performance, Architecture, Testing, Database, DevOps, Documentation et Code Quality.

4. Méthode d'analyse des constats

Le prototype lit automatiquement le fichier audit.csv grâce à la bibliothèque Pandas. Chaque constat est ensuite analysé selon sa catégorie. Une table de règles permet d'associer automatiquement :

- un niveau de criticité ;
- une priorité ;
- une estimation de charge ;
- une action corrective.

Les résultats sont enregistrés dans plusieurs fichiers CSV afin de faciliter leur exploitation par une équipe projet.

5. Critères de criticité et de priorité

La criticité est déterminée principalement en fonction de la catégorie du problème.

Catégorie	Criticité	Priorité
Security	Critique	P1
Performance	Élevée	P2
Testing	Élevée	P2
Architecture	Moyenne	P3
Database	Moyenne	P3
DevOps	Moyenne	P3
Code Quality	Faible	P4
Documentation	Faible	P4

Les problèmes de sécurité sont traités en priorité car ils représentent un risque important pour le système.

6. Plan de remédiation proposé

Sur la base de l'analyse automatique des constats, le prototype génère un plan de remédiation structuré. Chaque constat est associé à une action corrective concrète, classée par ordre de priorité.

ID	Catégorie	Action corrective	Priorité
AUD-001	Security	Mise en place d'un WAF et revue des ACL	P1
AUD-002	Security	Chiffrement des données sensibles en transit	P1
AUD-003	Performance	Optimisation des requêtes et mise en cache	P2

ID	Catégorie	Action corrective	Priorité
AUD-004	Testing	Couverture de tests unitaires > 80 %	P2
AUD-005	Architecture	Refactoring modulaire et découplage	P3
AUD-006	Database	Indexation et révision du schéma	P3
AUD-007	DevOps	Mise en place CI/CD et pipelines automatisés	P3
AUD-008	Code Quality	Revue de code et standards de codage	P4
AUD-009	Documentation	Rédaction de la documentation technique	P4

Ce plan constitue la base de travail pour l'équipe projet. Il est exporté automatiquement au format CSV sous le nom `remediation_plan.csv`.

7. Transformation en backlog

Le plan de remédiation est transformé en backlog agile. Chaque action corrective devient une user story associée à une priorité, une estimation de charge et un sprint de traitement.

ID	User Story	Priorité	Charge (j/h)	Sprint
AUD-001	Sécuriser les accès réseau	P1	3 j/h	Sprint 1
AUD-002	Chiffrer les données sensibles	P1	2 j/h	Sprint 1
AUD-003	Optimiser les performances	P2	4 j/h	Sprint 2
AUD-004	Augmenter la couverture de tests	P2	5 j/h	Sprint 2
AUD-005	Refactoring architecture	P3	6 j/h	Sprint 3
AUD-006	Optimisation base de données	P3	3 j/h	Sprint 3
AUD-007	Automatiser le déploiement CI/CD	P3	4 j/h	Sprint 4
AUD-008	Améliorer la qualité du code	P4	3 j/h	Sprint 4
AUD-009	Compléter la documentation	P4	2 j/h	Sprint 5

Le backlog est exporté dans le fichier `backlog.csv`, prêt à être intégré dans un outil de gestion de projet tel que Jira ou Azure DevOps.

8. Méthode d'estimation de charge

L'estimation de charge est réalisée automatiquement à partir d'une table de correspondance associant chaque catégorie à une fourchette de jours/homme. Cette méthode repose sur l'expérience empirique des équipes techniques.

Catégorie	Priorité	Charge estimée	Justification
Security	P1	3–5 j/h	Risque critique, intervention immédiate
Performance	P2	3–4 j/h	Impact utilisateur élevé
Testing	P2	4–5 j/h	Couverture tests à établir
Architecture	P3	5–7 j/h	Refactoring structurel
Database	P3	2–4 j/h	Optimisation schéma et index
DevOps	P3	3–5 j/h	Mise en place pipelines CI/CD
Code Quality	P4	2–3 j/h	Revue et normalisation du code
Documentation	P4	1–2 j/h	Rédaction guides et API docs

La charge totale estimée pour l'ensemble des constats est d'environ 32 jours/homme, répartis sur 5 sprints de deux semaines.

9. Gestion des dépendances et des risques

Certaines tâches présentent des dépendances fonctionnelles ou techniques. L'IA identifie ces liens à partir du champ `Depends_On` du fichier d'entrée et génère une matrice de dépendances.

ID	Tâche	Dépend de	Risque identifié
AUD-003	Optimisation performance	AUD-006	Blocage si DB non optimisée
AUD-005	Refactoring architecture	AUD-008	Régression si code non revu
AUD-007	CI/CD pipeline	AUD-004	Tests requis avant déploiement
AUD-004	Couverture de tests	AUD-005	Architecture instable = tests fragiles
AUD-009	Documentation	Toutes tâches	Doit refléter l'état final

La gestion des dépendances permet d'ordonner correctement les tâches dans la roadmap et d'éviter les blocages entre équipes.

10. Roadmap de traitement proposée

La roadmap est construite automatiquement en tenant compte des priorités, des estimations de charge et des dépendances identifiées. Elle propose un découpage en sprints de deux semaines.

Sprint	Période	Tâches	Objectif
Sprint 1	Semaines 1–2	AUD-001, AUD-002	Sécurisation critique
Sprint 2	Semaines 3–4	AUD-003, AUD-004	Performance & Tests
Sprint 3	Semaines 5–6	AUD-005, AUD-006	Architecture & DB
Sprint 4	Semaines 7–8	AUD-007, AUD-008	DevOps & Qualité
Sprint 5	Semaine 9	AUD-009	Documentation finale

La roadmap est exportée dans le fichier roadmap.csv. Elle offre une vision globale et planifiée du traitement de l'ensemble des constats d'audit.

11. Limites de la solution

Bien que le prototype soit fonctionnel, plusieurs limites doivent être prises en compte :

- Les estimations de charge sont basées sur des règles empiriques et peuvent ne pas refléter la complexité réelle de certains constats.
- La qualité du fichier d'entrée audit.csv conditionne directement la pertinence des résultats générés.
- Le système ne prend pas encore en compte l'historique des remédiation précédentes ou le contexte métier.
- La roadmap proposée est linéaire ; elle ne tient pas compte des contraintes de ressources humaines spécifiques à chaque organisation.
- L'absence de feedback loop ne permet pas à la solution d'apprendre et de s'améliorer automatiquement.

12. Ce que j'ai fait avec l'aide de l'IA

L'intelligence artificielle a été utilisée pour accélérer et automatiser plusieurs étapes du projet :

- Génération automatique des règles de criticité et de priorité à partir des catégories de constats.
- Proposition d'actions correctives adaptées à chaque type de problème identifié.
- Estimation de la charge de travail basée sur des tables de correspondance.
- Construction automatique du backlog priorisé et de la roadmap de traitement.
- Identification des dépendances entre les tâches à partir du champ Depends_On.
- Rédaction assistée de certains éléments de documentation et de synthèse.

L'IA a permis de réduire significativement le temps de traitement et d'assurer une cohérence dans la priorisation des constats.

13. Ce qui relève de mon travail personnel

La conception et la réalisation du prototype reposent sur plusieurs contributions personnelles :

- Définition de l'architecture globale de la solution et du flux de traitement des données.
- Conception de la table de règles associant catégories, criticités, priorités et actions correctives.
- Développement du script Python utilisant la bibliothèque Pandas pour la lecture et l'analyse du fichier CSV.
- Structuration du backlog selon les pratiques agiles et adaptation aux contraintes du projet.
- Validation et ajustement des estimations de charge générées.
- Rédaction du présent rapport et mise en forme de l'ensemble des livrables.
- Réflexion critique sur les limites de la solution et identification des pistes d'amélioration.

14. Conclusion et pistes d'amélioration

Conclusion

Le prototype développé démontre la viabilité d'une solution IA pour automatiser l'analyse d'audits informatiques et la génération de plans de remédiation. En combinant des règles métier structurées et le traitement automatique de données, la solution permet de produire en quelques secondes un backlog priorisé, une estimation de charge et une roadmap exploitable par une équipe projet.

Cette approche répond directement aux besoins des équipes d'audit qui cherchent à harmoniser leurs décisions et à réduire le temps de traitement des constats techniques.

Pistes d'amélioration

- Intégration d'un modèle de langage (LLM) pour analyser les descriptions textuelles des constats et affiner automatiquement la criticité.
- Mise en place d'un système de feedback permettant d'ajuster les estimations en fonction des réalisations passées.
- Ajout d'un module de visualisation graphique (Gantt, burndown chart) pour la roadmap.
- Connexion directe avec des outils de gestion de projet (Jira, Azure DevOps) via API.
- Prise en charge de formats d'entrée supplémentaires (Excel, JSON, PDF scanné via OCR).
- Développement d'une interface web permettant aux équipes non techniques d'utiliser la solution sans passer par la ligne de commande.

Annexes

Annexe A – Captures d'écran du prototype

Les captures d'écran suivantes illustrent l'exécution du prototype et les fichiers de sortie générés automatiquement.

Figure 1 – Exécution du prototype

```
Fichier chargé avec succès.
Nombre de constats : 50
-----
Analyse terminée.
Les fichiers ont été générés :
✓ summary.txt
✓ remediation_plan.csv
✓ backlog.csv
✓ roadmap.csv
-----
```

Figure 1 : Exécution du script `app.py` — analyse terminée et fichiers générés avec succès.

Figure 2 – Backlog généré

output >  backlog.csv						
	ID	Finding	Priorité	Charge (jours)	Depends_On	Status
4	C019	CSRF Protection Missing	P1	5	C018	Open
5	C018	XSS Vulnerability	P1	5		Open
6	C017	SQL Injection Risk	P1	5		Open
7	C043	Weak Encryption	P1	5		Open
8	C012	No HTTPS	P1	5	C011	Open
9	C011	Open Ports	P1	5		Open
10	C042	Secrets in Git	P1	5		Open
11	C002	Weak Password Policy	P1	5	C001	Open
12	C003	Outdated Dependency	P1	5		Open
13	C048	Session Timeout Missing	P1	5	C029	Open
14	C009	No Unit Tests	P2	3	C006	Open
15	C028	Large Files	P2	3		Open
16	C036	Cache Missing	P2	3		Open
17	C037	API Latency	P2	3	C036	Open
18	C010	Low Test Coverage	P2	3	C009	Open
19	C032	Regression Tests Missing	P2	3	C031	Open
20	C005	Missing Index	P2	3	C004	Open
21	C031	Integration Tests Missing	P2	3	C009	Open
22	C047	Manual Testing Only	P2	3		Open
23	C014	High CPU Usage	P2	3	C013	Open

Figure 2 : Backlog généré automatiquement par le prototype (`output/backlog.csv`).

Figure 3 – Plan de remédiation

output > remediation_plan.csv

	ID	Category	Finding	Criticité	Action corrective	Charge (jours)	Depends_On
1	C001	Security	Hardcoded Password	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	
2	C002	Security	Weak Password Policy	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	C001
3	C003	Security	Outdated Dependency	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	
4	C004	Performance	Slow SQL Query	Élevée	Optimiser les performances et réduire les temps de réponse.	3	
5	C005	Performance	Missing Index	Élevée	Optimiser les performances et réduire les temps de réponse.	3	C004
6	C006	Code Quality	Duplicated Code	Faible	Refactoriser le code et supprimer les duplications.	1	
7	C007	Code Quality	Large Function	Faible	Refactoriser le code et supprimer les duplications.	1	C006
8	C008	Documentation	Missing Documentation	Faible	Compléter la documentation technique.	2	
9	C009	Testing	No Unit Tests	Élevée	Mettre en place des tests automatisés.	3	C006
10	C010	Testing	Low Test Coverage	Élevée	Mettre en place des tests automatisés.	3	C009
11	C011	Security	Open Ports	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	
12	C012	Security	No HTTPS	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	C011
13	C013	Performance	Memory Leak	Élevée	Optimiser les performances et réduire les temps de réponse.	3	
14	C014	Performance	High CPU Usage	Élevée	Optimiser les performances et réduire les temps de réponse.	3	C013
15	C015	Architecture	Tight Coupling	Moyenne	Revoir l'architecture afin de réduire le couplage.	2	
16	C016	Architecture	No Layer Separation	Moyenne	Revoir l'architecture afin de réduire le couplage.	2	C015
17	C017	Security	SQL Injection Risk	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	
18	C018	Security	XSS Vulnerability	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	
19	C019	Security	CSRF Protection Missing	Critique	Corriger la vulnérabilité et appliquer les bonnes pratiques de sécurité.	5	C018
20	C020	DevOps	No CI/CD	Moyenne	Automatiser le déploiement et renforcer la supervision.	2	
21	C021	DevOps	No Monitoring	Moyenne	Automatiser le déploiement et renforcer la supervision.	2	
22	C022	DevOps	No Logging	Moyenne	Automatiser le déploiement et renforcer la supervision.	2	
23	C023	Database	Null Values	Moyenne	Améliorer la qualité et l'intégrité des données.	2	
24	C024	Database	Duplicate Records	Moyenne	Améliorer la qualité et l'intégrité des données.	2	C023
25	C025	Database	Inconsistent Dates	Moyenne	Améliorer la qualité et l'intégrité des données.	2	C023
26	C026	Code Quality	Magic Numbers	Faible	Refactoriser le code et supprimer les duplications.	1	
27	C027	Code Quality	Unused Variables	Faible	Refactoriser le code et supprimer les duplications.	1	

Figure 3 : Plan de remédiation généré automatiquement (output/remediation_plan.csv).

Figure 4 – Roadmap

output > roadmap.csv

	Sprint	Nombre de tâches	Charge totale
1	P1	13	65
2	P2	13	39
3	P3	16	32
4	P4	8	8
5			

Figure 4 : Roadmap de traitement proposée par le prototype (output/roadmap.csv).